

Laboratorio #01
Sockets TCP y UDP
Ciencias de la Computación VIII

Resumen:

Desarrollar una aplicación cliente/servidor que utilice sockets TCP y UDP en Java para realizar operaciones matemáticas básicas. El cliente solicitará al usuario una operación matemática en forma de cadena de caracteres (por ejemplo, $2+3$ o $5*6$), se conectará al servidor y le enviará la cadena. El servidor procesará la cadena, realizará la operación y devolverá el resultado al cliente. Ambos programas deben manejar errores de entrada y solo terminar la conexión con EXIT.

Instrucciones

Preparación:

- Lea los **RFC 793 (TCP)** y **RFC 768 (UDP)** para comprender los protocolos.
- Investigue la diferencia entre TCP y UDP y cuándo utilizar cada uno.

Implementación

Se implementaran 2 Clases con los siguientes nombres:

- **Client.java**
- **Server.java**

Para definir cómo se va a ejecutar se tomarán argumentos desde la línea de comandos en formato **KEY VALUE** sin importar el orden de aparición.

- **protocol:** variable que contendrá el tipo de protocolo a ejecutar
- **server:** IP del Servidor al cual conectarse.
- **port:** Puerto en el que estará escuchando el servidor.

Ejemplos de ejecución en línea de comandos:

```
./javac Client.java
./java Client protocol UDP server 192.168.100.55 port
3001

./javac Server.java
./java Server port 3001 protocol UDP

./javac Client.java
./java Client port 5030 server 192.168.100.30 protocol
TCP

./javac Server.java
./java Server protocol TCP port 5030
```

Funcionalidad

Cliente:

- Conectarse al servidor correspondiente (TCP o UDP).
- Solicitar al usuario una cadena con una operación matemática (por ejemplo, $2+3$, $5-2$, $4*7$, $8/2$, $8\%2$).
- Enviar la cadena al servidor y esperar la respuesta.
- Mostrar el resultado de la operación.
- Continuar hasta que el usuario ingrese **EXIT**.

Servidor:

- Esperar conexiones de clientes (TCP) o paquetes de datos (UDP).
- Leer la cadena de operación, realizar la operación matemática y devolver el resultado al cliente.
- Continuar funcionando hasta que reciba **EXIT**.

NOTA: En TCP mantener la conexión hasta que se envíe la palabra **EXIT**, en UDP no es necesario ya que no está orientado a la conexión.

Formato de Log:

Utilice el siguiente formato de log en base a CLF:

[s] host socketType [dateTime] protocol: description

[s]: Sentido del envío utilizando "<" Sale o ">" Entra

host: IP del cliente o servidor

socketType: client o server

dateTime: En el formato yyyy/mm/dd hh:mm:ss

protocol: UDP o TCP

description: Es request o response según la acción.

Ejemplo de log visto desde el lado cliente:

```
Ingrese la operación a realizar: 8%5
< 192.168.1.2 client [10/07/2024 15:20:30] TCP: 8%5
> 192.168.1.5 server [10/07/2024 15:20:31] TCP: 3
Ingrese la operación a realizar: EXIT
< 192.168.1.2 client [10/07/2024 15:21:10] TCP: EXIT
> 192.168.1.5 server [10/07/2024 15:21:12] TCP: EXIT
```

Ejemplo de log visto desde el lado servidor:

```
> 192.168.1.2 client [10/07/2024 15:20:30] TCP: 8%5
< 192.168.1.5 server [10/07/2024 15:20:31] TCP: 3
> 192.168.1.2 client [10/07/2024 15:21:10] TCP: EXIT
< 192.168.1.5 server [10/07/2024 15:21:12] TCP: EXIT
```

Nota: El sentido de envío en el lado del servidor el cliente entra ">", si vemos en el lado del cliente el servidor es el que entra ">".

- Asegúrese de manejar excepciones y errores de entrada.
- Maneje valores por defecto en dado caso no se especifique valores en los argumentos.
- Implemente la funcionalidad de logs para ambos protocolos

Entrega

- Suba un archivo ZIP al GES con todos los archivos fuente.
- La calificación se realizará de manera presencial, en dado caso que no pueda presentarse enviar un video Explicando su Código, mostrando la compilación y ejecución, hacer varios ejemplos de su Funcionalidad leyendo los LOGs.
- Use Java 21 o 22 para su implementación.
- El laboratorio puede tener una calificación de cero si:
 - Si no compila (en java 21 o 22)
 - Si los LOGS no tienen el formato establecido.
 - Si se detecta plagio o copia.
 - Si no vienen las 2 Clases especificadas.
 - Si no siguió las instrucciones del funcionamiento.